

Topic 5: Additional Predictive Modeling Techniques

Instructor: Dr. Akash Gupta

Table of contents

1	Multiple Linear Regression Model	2
1.1	Introduction	2
1.2	Real-World Examples	2
1.3	How It Works (Conceptually)	2
1.4	Modeling	3
1.5	Interpreting Coefficients	3
1.6	Model Evaluation	3
1.7	Common Pitfalls to Watch For	4
1.8	Implementation in Python	4
2	Polynomial Regression	6
2.1	Introduction	6
2.2	Modeling	6
2.3	Implementation in Python	7
3	Support Vector Machines (SVM)	8
3.1	Introduction	8
3.2	Hyperplane and Margin	9
3.3	Support Vectors	9
3.4	Kernel	9
3.5	Kernel Trick	10
3.6	Python Implementation	10
4	XGBoost (Extreme Gradient Boosting)	12
4.1	Introduction	12
4.2	Modeling	12
4.3	Python Implementation	13

1 Multiple Linear Regression Model

1.1 Introduction

Multiple Linear Regression (MLR)** is an extension of simple linear regression that models the relationship between one **dependent variable** (continuous outcome) and two or more **independent variables** (predictors).

Think of it as a smarter version of a simple rule of thumb. Instead of saying “bigger houses cost more,” MLR lets you say: “a house’s price depends on its size, the number of bedrooms, the neighborhood, and the year it was built — all at the same time.”

Key Idea: MLR finds the best-fit formula that uses multiple inputs (predictors) to predict one output (outcome).

1.2 Real-World Examples

Business Problem	Outcome (Y)	Predictors (X's)
Forecasting sales	Monthly revenue	Ad spend, promotions, seasonality
Credit risk	Loan default probability	Income, credit score, debt-to-income ratio
HR analytics	Employee attrition	Tenure, salary, engagement scores
Healthcare	Hospital readmission risk	Age, diagnosis, # of prior visits

1.3 How It Works (Conceptually)

Imagine you’re a real estate analyst trying to price homes. You gather data on recent sales and notice that price depends on:

- Square footage
- Number of bedrooms
- Distance from downtown

MLR doesn’t just look at each factor separately — it weighs them *together* and finds the combination that best explains the price differences you observe. The result is a formula like:

$$\text{Predicted Price} = \text{Base Value} + (\text{Weight} \times \text{Square Footage}) + (\text{Weight} \times \text{Bedrooms}) + (\text{Weight} \times \text{Distance})$$

Each “weight” (called a **coefficient**) tells you how much that factor matters, while holding everything else constant.

1.4 Modeling

For p predictors, the MLR model is:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$$

- Y : Dependent variable (outcome).
- β_0 : Intercept (value of Y when all $X_i = 0$).
- $\beta_1, \beta_2, \dots, \beta_p$: Coefficients (slopes) for each predictor ($X_1, X_2, \dots, X_p$).
- ϵ : Error term

Use the **Ordinary Least Squares (OLS)** method to minimize the sum of squared residuals. This is how we estimate coefficient of each predictor can be estimated.

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}))^2$$

1.5 Interpreting Coefficients

- **Intercept:** The baseline prediction when all predictors are at zero (often not meaningful on its own).
- **Coefficients:** The “lever” for each predictor. For example, if the coefficient for square footage is 100, it means *each additional square foot adds \$100 to the predicted price, all else equal*.

1.6 Model Evaluation

- **R-squared:** How much of the variation in your outcome the model explains. An R^2 of 0.85 means 85% of the price differences are captured by your predictors — that’s strong.

- **Adjusted R-squared:** Similar to R^2 , but penalizes you for throwing in too many predictors that don't actually help. Use this when comparing models with different numbers of variables.
- **F-test:** Tests overall significance of the model.
- **t-tests:** Test significance of individual coefficients.

1.7 Common Pitfalls to Watch For

- **Correlation $\neq$ Causation.** Just because ad spend correlates with sales doesn't mean spending more *causes* more sales. There could be confounding factors (e.g., you tend to advertise more during peak seasons anyway).
- **Overfitting.** Adding too many predictors can make your model fit the training data beautifully but fail on new data. Use adjusted R^2 and validation to guard against this.
- **Multicollinearity.** If your predictors are highly correlated with each other (e.g., square footage and number of rooms), the model can get confused about which one is doing the work.

1.8 Implementation in Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split
from mpl_toolkits.mplot3d import Axes3D # For 3D plotting

# Set random seed for reproducibility
np.random.seed(42)

# Step 1: Generate synthetic data
n_samples = 100
X1 = np.random.uniform(1000, 3000, n_samples) # Predictor 1: e.g., square footage
X2 = np.random.uniform(1, 5, n_samples)         # Predictor 2: e.g., number of bedrooms
X = np.column_stack((X1, X2))                   # Combine into a 2D array

# True relationship: y = 50000 + 100*X1 + 20000*X2 + noise
y = 50000 + 100 * X1 + 20000 * X2 + np.random.normal(0, 10000, n_samples)
```

```
# Step 2: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 80% train (80 samples), 20% test (20 samples)

# Step 3: Fit the Multiple Linear Regression model on training data
model = LinearRegression()
model.fit(X_train, y_train)

# Step 4: Make predictions
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)

# Step 5: Evaluate the model
r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)

# Step 6: Print results
print("Model Coefficients:")
print(f"Intercept: {model.intercept_:.2f}")
print(f"Coefficient for X1: {model.coef_[0]:.2f}")
print(f"Coefficient for X2: {model.coef_[1]:.2f}")
print(f"R-squared (Training): {r2_train:.4f}")
print(f"R-squared (Testing): {r2_test:.4f}")
```

```
Model Coefficients:
Intercept: 47214.53
Coefficient for X1: 98.75
Coefficient for X2: 21967.93
R-squared (Training): 0.9737
R-squared (Testing): 0.9856
```

Reading the Output: The model recovered the true relationships closely. Each additional square foot adds ~\$99 to price, and each bedroom adds ~\$22,000 — very close to the values we used to generate the data. The R^2 of 0.99 tells us the model explains nearly all price variation.

2 Polynomial Regression

2.1 Introduction

- Polynomial Regression is an extension of **Linear Regression** that models non-linear relationships between the independent variable(s) (X) and the dependent variable (Y).
- Instead of fitting a straight line ($Y = \beta_0 + \beta_1 X$), it fits a polynomial curve (e.g., $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_n X^n$).
- The *polynomial* refers to the powers of (X) (e.g., X^2 , X^3) added as features.
- Linear regression assumes a straight-line relationship, but real-world data often shows curves or non-linear patterns.
- Examples:
 - The growth rate of plants vs. sunlight (diminishing returns).
 - Car speed vs. fuel efficiency- In the car speed versus fuel efficiency example, fuel efficiency improves as speed increases up to a certain point (the peak), then drops off as speed continues to rise.

For a single variable X, the polynomial regression equation is:

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots + \beta_n X^n + \epsilon$$

- β_0 : Intercept.
- $\beta_1, \beta_2, \dots, \beta_n$: Coefficients for each term.
- n : Degree of the polynomial (e.g., $n = 2$ for quadratic, $n = 3$ for cubic).
- ϵ : Error term.

2.2 Modeling

1. **Transform the Features:** Add polynomial terms (e.g., X^2 , X^3) as new features.
2. **Fit a Linear Model:** Use the same least squares method as linear regression to find the best β values.
3. **Predict:** Plug in X values to get a prediction.

2.3 Implementation in Python

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score

# Set random seed for reproducibility
np.random.seed(42)

# Step 1: Generate synthetic data
X = np.linspace(0, 10, 100).reshape(-1, 1) # 100 points from 0 to 10
y = 2 + 3 * X - 0.5 * X**2 + np.random.normal(0, 2, (100, 1)) # Add some noise

# Step 2: Transform the data to include polynomial features (degree 2)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X) # Creates [1, X, X^2] as features

# Step 3: Fit the polynomial regression model
model = LinearRegression()
model.fit(X_poly, y)

# Step 4: Make predictions
y_pred = model.predict(X_poly)

# Step 5: Calculate R-squared
r2 = r2_score(y, y_pred)
print(f"R-squared: {r2:.4f}")

# Step 6: Visualize the results
plt.scatter(X, y, color='blue', label='Data points')
plt.plot(X, y_pred, color='red', label='Polynomial fit (degree 2)', linewidth=2)
plt.xlabel('X')
plt.ylabel('y')
plt.title('Polynomial Regression')
plt.legend()
plt.show()
```

R-squared: 0.9350

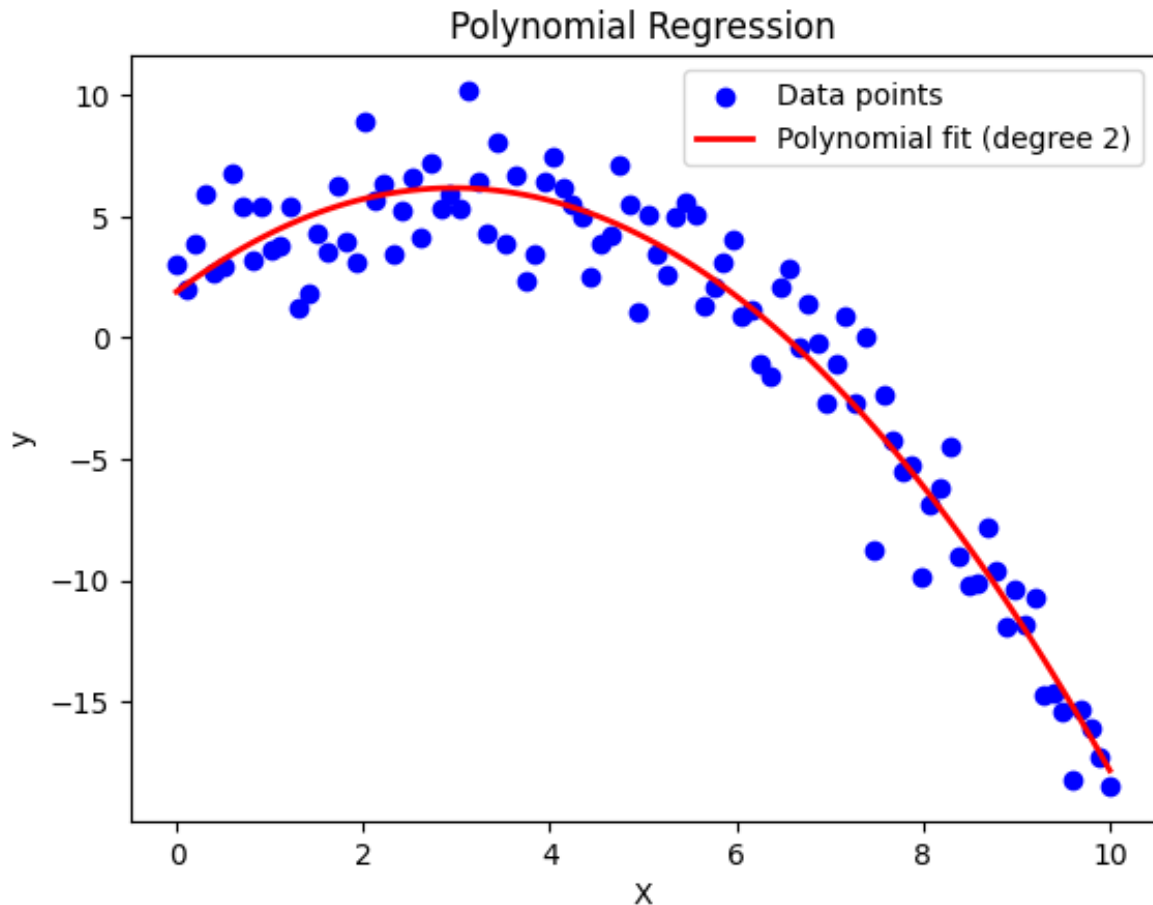


Figure 1: Polynomial Regression Model

- Use cross-validation or metrics like (R^2), RMSE to test different degrees.

3 Support Vector Machines (SVM)

3.1 Introduction

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that best separates the data points in a high-dimensional space. SVM is particularly effective in high-dimensional spaces and is widely used for both linear and non-linear classification problems.

3.2 Hyperplane and Margin

- The goal of SVM is to find the hyperplane that maximizes the margin between different classes.
- The margin is the distance between the hyperplane and the closest data points from each class (support vectors).
- A larger margin reduces the model's generalization error.

3.3 Support Vectors

- Support vectors are the data points closest to the hyperplane.
- These points influence the position and orientation of the hyperplane.

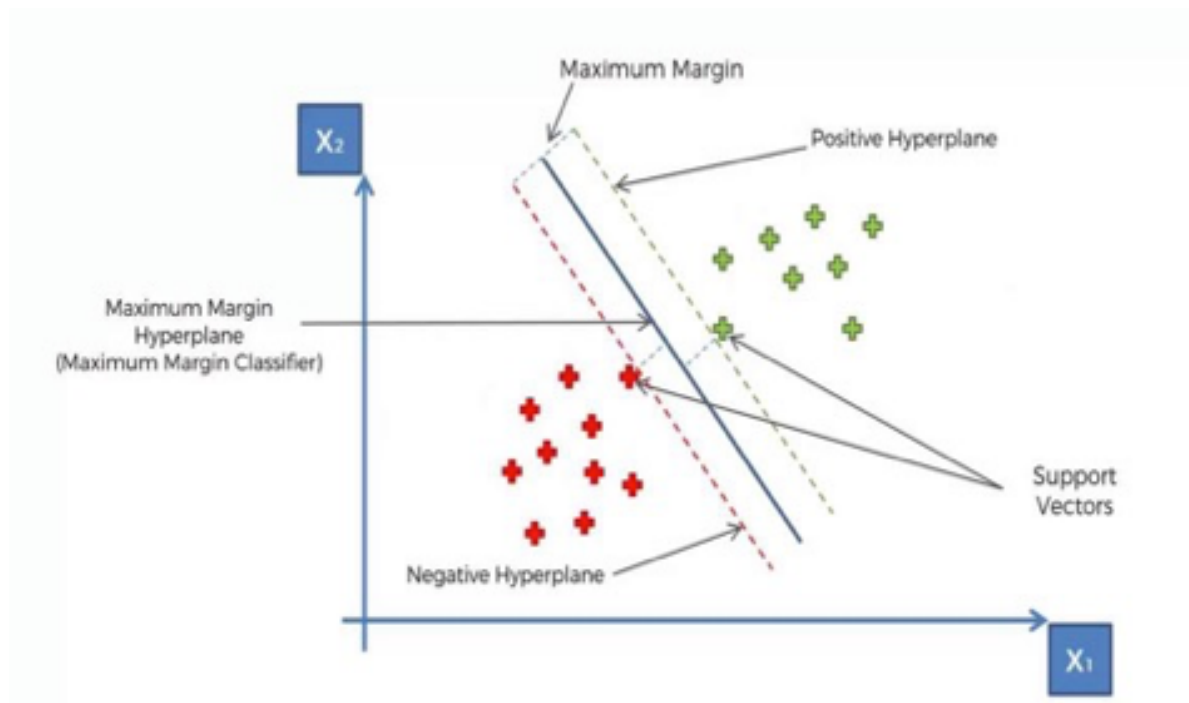


Figure 2: Source: Internet

3.4 Kernel

A kernel is a function that computes a similarity between two data points.

Linear Kernel: $K(x, x') = x^T x'$ — simple dot product.

Polynomial Kernel $K(x, x') = (x^T x' + c)^d$ — captures polynomial relationships.

RBF (Gaussian) Kernel

$$K(x, x') = \exp(-\gamma |x - x'|^2)$$

— measures similarity based on distance.

3.5 Kernel Trick

A kernel is a mathematical function that transforms data into a higher-dimensional space to make it easier to separate using a linear boundary.

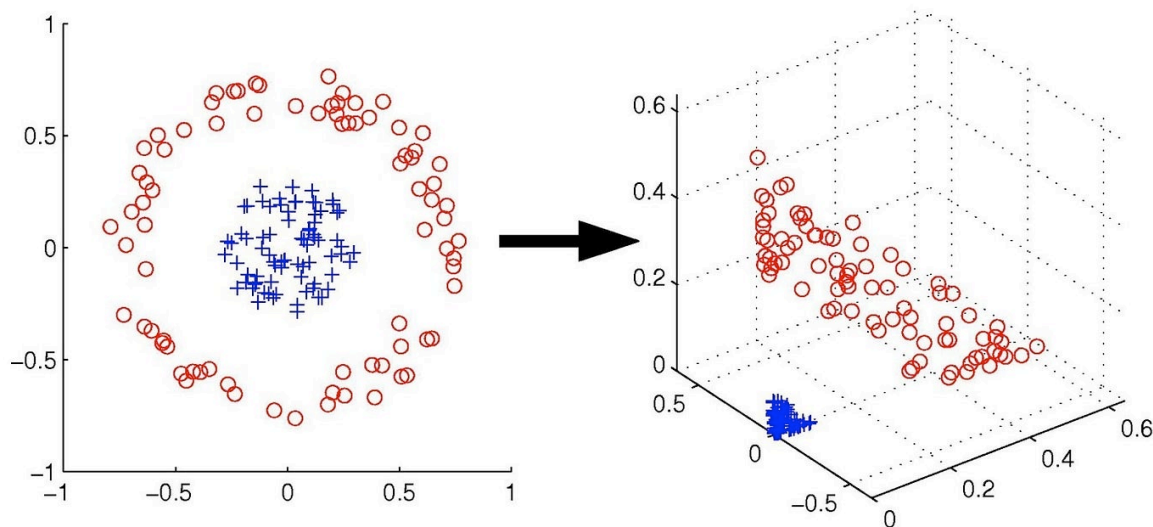


Figure 3: Example of a labeled data linearly inseparable in 2-Dimension is linearly separable in 3-Dimension. Source: Internet

3.6 Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
```

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, \
    confusion_matrix, classification_report

# Generate synthetic data
X, y = make_classification(n_samples=500, n_features=2, n_classes=2, \
    n_clusters_per_class=1, n_redundant=0, random_state=42)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train an SVM classifier
svm_model = SVC(kernel='rbf')
svm_model.fit(X_train, y_train)

# Make predictions
y_pred = svm_model.predict(X_test)

# Compute performance metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# Display results
print("Performance Metrics:")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")
```

```
Performance Metrics:
Accuracy: 0.93
Precision: 0.88
Recall: 1.00
F1 Score: 0.93
```

4 XGBoost (Extreme Gradient Boosting)

4.1 Introduction

XGBoost (Extreme Gradient Boosting) is an optimized, scalable, and efficient implementation of gradient boosting, a machine learning technique for regression and classification.

- High performance and speed (parallelized tree building).
- Handles missing data automatically.
- Regularization to prevent overfitting.
- Supports custom loss functions.

4.2 Modeling

- **Idea:** Builds an ensemble of weak learners (typically decision trees) sequentially, where each tree corrects the errors of the previous ones.

- **Steps:**

1. Start with an initial prediction (e.g., mean for regression).
2. Compute residuals (errors) between predictions and actual values.
3. Fit a new tree to the residuals.
4. Update predictions by adding the new tree's output (scaled by a learning rate).

New Prediction = Previous Prediction + (Learning Rate * Tree Output)

5. Repeat until a stopping criterion is met (e.g., max trees or no improvement).

- **Mathematical Formula:**

$Obj = Loss + Regularization$

- **Loss:** Measures prediction error (e.g., MSE for regression).
- **Regularization:** Penalizes complex models (e.g., Number of leaves, L2 regularization term)

Key Hyperparameters

- **Learning Rate (eta):** Scales the contribution of each tree (0.01–0.3 typical).

- **Max Depth:** Maximum depth of each tree (controls overfitting, 3–10 common).
- **Subsample:** Fraction of data sampled for each tree (e.g., 0.8).
- **Gamma:** Minimum loss reduction required to make a split.
- **Lambda/Alpha:** L2/L1 regularization strength.

4.3 Python Implementation

```
import numpy as np
import xgboost as xgb
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

# Generate synthetic regression data
X, y = make_regression(
    n_samples=1000,      # 1000 data points
    n_features=5,         # 5 features
    noise=10,             # Add some noise
    random_state=42       # For reproducibility
)

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Define and train the XGBoost model using XGBRegressor
model = xgb.XGBRegressor(
    max_depth=3,          # Max depth of trees
    learning_rate=0.1,     # Learning rate (eta)
    n_estimators=100,      # Number of trees (replaces num_boost_round)
    eval_metric='rmse',    # Root mean squared error for evaluation
    random_state=42       # For reproducibility
)

model.fit(X_train, y_train,)
```

```
# Make predictions on test set
y_pred = model.predict(X_test)

# Evaluate performance
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
print(f"\nFinal RMSE on Test Set: {rmse:.2f}")

# Visualize predictions vs actual values
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("XGBoost (XGBRegressor): Actual vs Predicted")
plt.show()
```

Final RMSE on Test Set: 16.21

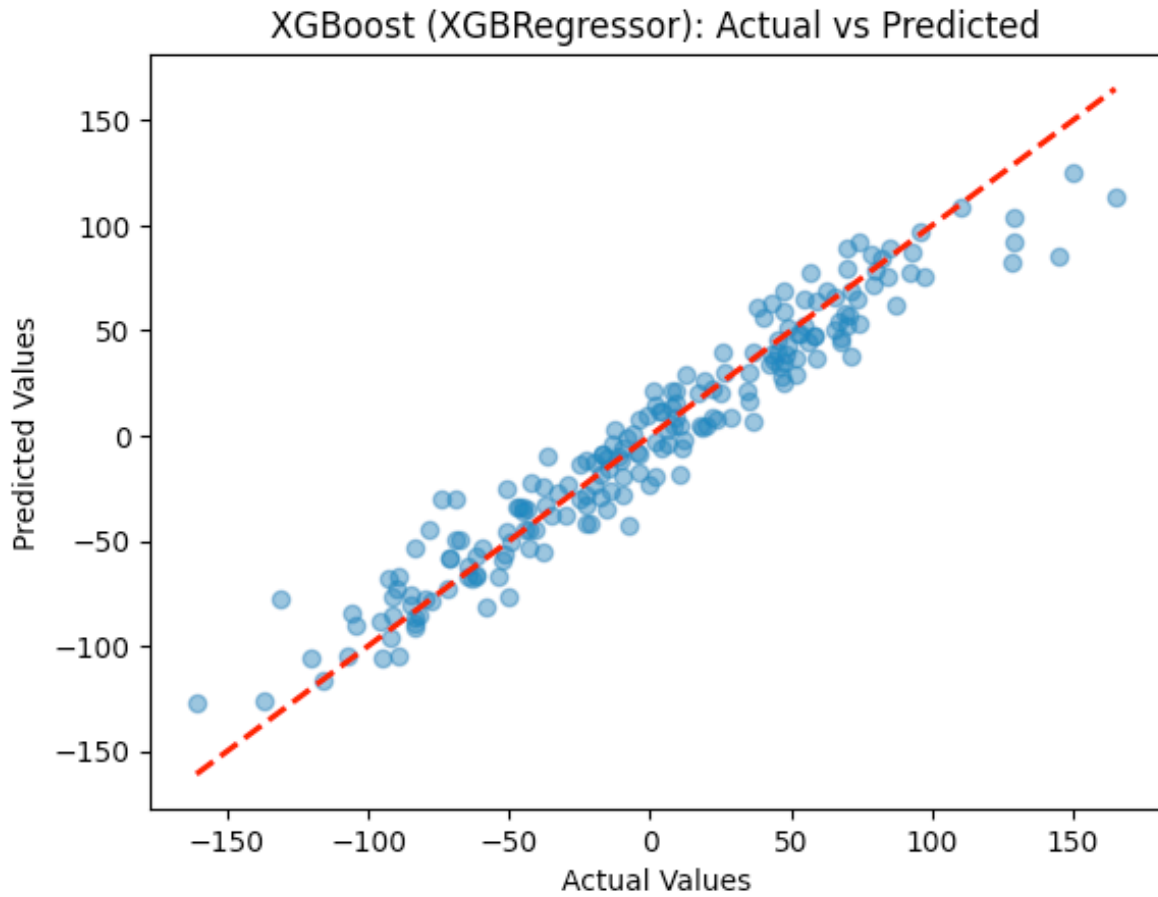


Figure 4: XGBoost